

Experiment No. 7

Experiment Name - Motion Estimation using Optical Flow Algorithms in Video Sequences

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Import the required Python libraries and load a video sequence or access a live webcam stream using OpenCV.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode

def take_photo():
    js = Javascript('''
    async function takePhoto() {
        const div = document.createElement('div');

        const video = document.createElement('video');
        video.style.display = 'block';
        video.width = 640;
        video.height = 480;

        const stream = await navigator.mediaDevices.getUserMedia({video: true});

        document.body.appendChild(div);
        div.appendChild(video);

        video.srcObject = stream;
        await video.play();

        await new Promise((resolve) => setTimeout(resolve, 1000));

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;

        canvas.getContext('2d').drawImage(video, 0, 0);

        stream.getVideoTracks()[0].stop();
        div.remove();

        return canvas.toDataURL('image/jpeg');
    }
    ''')
    display(js)

def get_frame():
    data = eval_js('takePhoto()')
    binary = b64decode(data.split(',')[1])
    image = np.frombuffer(binary, dtype=np.uint8)
    frame = cv2.imdecode(image, cv2.IMREAD_COLOR)
    return frame

take_photo() # Call take_photo() to define the javascript function
frame1 = get_frame()

print("Webcam frame captured successfully!")

plt.figure(figsize=(5, 4))
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Live Webcam Frame")
plt.axis("off")
plt.show()
```

Webcam frame captured successfully!

Live Webcam Frame



2. Read consecutive video frames and convert them to grayscale for optical flow computation.

```
import time

take_photo() # Ensure the JavaScript function is defined
frame1 = get_frame()
time.sleep(0.5)
frame2 = get_frame()

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

print("Two consecutive webcam frames captured.")

plt.figure(figsize=(8, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray1, cmap="gray")
plt.title("Grayscale Frame")
plt.axis("off")

plt.show()
```

Two consecutive webcam frames captured.

Frame 1



Grayscale Frame



3. Implement the Lucas-Kanade Optical Flow algorithm to estimate the motion of selected feature points across consecutive frames.

```
# Detect feature points
p0 = cv2.goodFeaturesToTrack(
    gray1,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

print("Feature points detected:", len(p0))

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
```

```

    )

# Calculate optical flow
p1, status, error = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

# Select successfully tracked points
good_new = p1[status == 1]
good_old = p0[status == 1]

print("Successfully tracked points:", len(good_new))

```

```

Feature points detected: 70
Successfully tracked points: 70

```

4. Visualize the motion vectors by drawing arrows or trajectories representing the movement of tracked points.

```

lk_output = frame2.copy()

for new, old in zip(good_new, good_old):

    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tiplength=0.3
    )

    cv2.circle(
        lk_output,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(6, 4))
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()

```

Lucas-Kanade Sparse Optical Flow



5. Implement the Farneback Dense Optical Flow algorithm to estimate motion for every pixel in the image.

```

flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

```

```
)

print("Farneback Dense Optical Flow calculated.")
print("Flow shape:", flow.shape)
```

```
Farneback Dense Optical Flow calculated.
Flow shape: (480, 640, 2)
```

6. Display the dense optical flow output using color-coded motion visualization to represent direction and magnitude.

```
# Convert flow into magnitude and angle
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV image
hsv = np.zeros_like(frame2)

hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 1] = 255
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert HSV to BGR
dense_output = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

plt.figure(figsize=(7, 4))
plt.imshow(cv2.cvtColor(dense_output, cv2.COLOR_BGR2RGB))
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```

Farneback Dense Optical Flow



7. Compare the performance of Lucas-Kanade and Farneback algorithms in terms of accuracy, computational complexity, and robustness.

```
plt.figure(figsize=(7, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(dense_output, cv2.COLOR_BGR2RGB))
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.show()
```

Lucas-Kanade Sparse Optical Flow



Farneback Dense Optical Flow



✓ Comparison of Lucas-Kanade and Farneback

Lucas-Kanade:

- Sparse optical flow method.
- Tracks selected feature points.
- Generally faster.
- Useful for feature and object tracking.

Farneback:

- Dense optical flow method.
- Estimates motion over a large number of pixels.
- Provides detailed motion information.
- More computationally expensive.

8. Analyze the effect of object speed, lighting conditions, and camera motion on optical flow estimation.

```
average_motion = np.mean(magnitude)
maximum_motion = np.max(magnitude)

print("Average Motion Magnitude:", average_motion)
print("Maximum Motion Magnitude:", maximum_motion)
```

```
Average Motion Magnitude: 0.47983262
Maximum Motion Magnitude: 5.0560713
```

Effect of different conditions:

Analyzing Speed, Lighting and Camera Motion:

1. **Object Speed:** Faster movement generally produces larger optical-flow vectors, while slow movement produces smaller vectors.
2. **Lighting:** Sudden changes in lighting may produce incorrect or false motion.
3. **Camera Motion:** Moving the camera can produce optical flow across a large part of the image.

9. Evaluate the suitability of each algorithm for different real-world applications such as object tracking, surveillance, and autonomous navigation.

Applications of Optical Flow are:

1. **Object Tracking:** Lucas-Kanade can track selected feature points on moving objects.
 2. **Video Surveillance:** Farneback can detect motion over larger regions of a scene.
 3. **Gesture Recognition:** Optical flow can estimate hand and body movements.
 4. **Sports Analytics:** Dense optical flow can help analyze player and object movement.
 5. **Autonomous Navigation:** Dense optical flow can provide information about scene and camera movement.
 6. **Robotics:** Optical flow can help robots understand movement in their surroundings.
10. Summarize the observations and discuss the significance of optical flow in dynamic scene analysis and motion-based computer vision systems.

OPTICAL FLOW EXPERIMENT - FINAL SUMMARY, OBSERVATIONS AND CONCLUSION

Lucas-Kanade Sparse Optical Flow: Tracks selected feature points between consecutive video frames.

Farneback Dense Optical Flow: Estimates motion for a large number of pixels in the image.

Motion Results:

- Average Motion Magnitude: Obtained from the optical-flow calculation.
- Maximum Motion Magnitude: Obtained from the optical-flow calculation.

Observations:

- 1. Optical flow estimates apparent motion between consecutive video frames.
- 2. Lucas-Kanade provides sparse motion vectors.
- 3. Farneback provides dense motion information.
- 4. Faster movement generally produces larger motion vectors.
- 5. Lighting changes can affect optical-flow estimation.
- 6. Camera movement can create motion across the entire scene.
- 7. Optical flow is useful in object tracking, surveillance, robotics, gesture recognition, sports analytics, and autonomous navigation.

Conclusion: Optical Flow is an important computer vision technique for estimating motion and understanding dynamic scenes in video.

Questions and Answers

1. What is Optical Flow? How is it used in computer vision?

Optical Flow is the apparent motion of objects or pixels between two consecutive video frames. It represents the **direction and speed of movement** using motion vectors. It is used in object tracking, video surveillance, robotics, autonomous driving, and action recognition.

2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Lucas-Kanade is a **sparse optical flow** algorithm that tracks selected feature points between two consecutive frames.

Working:

- 1. Detect important feature points.
- 2. Compare their positions in consecutive frames.
- 3. Assume that the motion is small and brightness remains approximately constant.
- 4. Calculate the displacement of the feature points.
- 5. Represent the motion using vectors or arrows.

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense Optical Flow estimates motion for almost every pixel in an image, whereas **Sparse Optical Flow** estimates motion only for selected feature points.

Sparse Optical Flow	Dense Optical Flow
Tracks selected feature points	Estimates motion across the image
Generally faster	More computationally expensive
Example: Lucas-Kanade	Example: Farneback
Useful for feature tracking	Useful for detailed motion analysis

4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Lucas-Kanade	Farneback
Sparse optical flow	Dense optical flow
Tracks selected feature points	Estimates motion for many pixels
Generally faster	Generally more computationally expensive
Useful for object and feature tracking	Useful for detailed motion analysis
Uses local image information	Uses polynomial approximation of image neighborhoods

5. What assumptions are made while computing optical flow?

The main assumptions are:

- 1. **Brightness Constancy:** The brightness of a pixel remains approximately constant between consecutive frames.
- 2. **Small Motion:** The movement between consecutive frames is relatively small.
- 3. **Spatial Smoothness:** Nearby pixels generally have similar motion.
- 4. **Continuous Motion:** Motion changes smoothly over small regions.

6. What factors can affect the accuracy of optical flow estimation?

The accuracy of optical flow can be affected by:

- Fast or large object movement
- Sudden changes in lighting
- Shadows and reflections
- Motion blur
- Camera movement
- Occlusion of objects
- Low-texture regions
- Noise in images

7. Mention five real-world applications of optical flow in computer vision.

Five important applications are:

1. **Object Tracking**
2. **Video Surveillance**
3. **Autonomous Driving**
4. **Gesture and Action Recognition**
5. **Sports Analytics**

Other applications include robotics, augmented reality, and video stabilization.

8. Why are grayscale images generally used for optical flow computation?

Grayscale images are generally used because optical flow mainly depends on **intensity changes** between frames. Grayscale reduces the image from three color channels to one channel, making the computation simpler and faster while still providing useful motion information.

9. What are the limitations of optical flow algorithms in real-world environments?

Some important limitations are:

- Sensitive to sudden lighting changes.
- Difficult to estimate very fast or large motion accurately.
- Camera motion can create unwanted optical flow.
- Occlusion can cause incorrect motion estimation.
- Low-texture regions provide little motion information.
- Noise and motion blur can reduce accuracy.
- Dense optical flow can require high computational resources.

10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition?

- **Autonomous Driving:** Optical flow helps estimate the movement of vehicles, pedestrians, and the surrounding scene.
- **Video Surveillance:** It helps detect and track moving objects or people.
- **Action Recognition:** It helps identify movements such as walking, running, waving, and other human actions.

Thus, optical flow provides important **motion information from consecutive video frames** for understanding dynamic scenes.